Université de Haute-Alsace
École Nationale Supérieure d'Ingénieurs
Sud Alsace

Informatique et Réseaux - 2A

Artificial Intelligence and Reinforcement Learning

TD3 : Game Search

Introduction

In this practical session, you will implement and compare three game-search algorithms studied in the lectures: **Minimax**, **Alpha-Beta Pruning**, and **Monte Carlo Tree Search (MCTS)**.

You will apply these algorithms to **Connect Four**, a two-player game played on a 6×7 grid. Players take turns dropping discs into columns, and the first player to align four discs horizontally, vertically, or diagonally wins.

The objective is to develop an AI agent capable of playing against a human opponent and to investigate how different search strategies affect its decisions and computational cost.

Getting Started

The source code for this practical session is available as a ZIP archive:

`https://hadifawaz1999.github.io/Artificial-Intelligence-Reinforcement-Learning--2A-Info/TD/TD_03_Game_Search/source_code.zip`

Download and extract the archive, then navigate to the `code/` directory.

Install the required dependencies:

```
python -m pip install -r requirements.txt
```

Launch the Flask application:

```
python app.py
```

Open `http://localhost:5000` in your browser.

Project Structure

The application consists of the following main Python files:

- `game.py`: Implements the rules of Connect Four, including legal moves, winning conditions, and a heuristic evaluation function.
- `algorithms.py`: Contains the implementations of Minimax, Alpha-Beta Pruning, and MCTS. **This is the only file you need to modify.**
- `app.py`: Connects the algorithms to the Flask web application.

Search Visualization

The web application provides an interactive visualization of the search process performed by the AI.

After each AI move, the corresponding search tree can be explored using the visualization panel. The playback controls allow you to navigate through the search process step by step, or automatically replay it as an animation.

Depending on the selected algorithm, the visualization illustrates different aspects of the search:

- **Minimax:** Observe how the game tree is explored, how MAX and MIN nodes alternate, and how evaluation scores propagate through the tree.
- **Alpha-Beta Pruning:** Observe the same Minimax exploration, including updates to the α and β bounds and the branches that are pruned because they cannot influence the final decision.
- **MCTS:** Observe how the search tree progressively expands through simulations, how visit counts and estimated rewards evolve, and how UCT guides the exploration of promising actions.

For Minimax and Alpha-Beta, the **search depth** controls how many moves ahead the AI explores. For MCTS, the **number of simulations** controls how many search iterations are performed.

Each node represents a possible game state. The number displayed at the top of a node indicates the column played to reach that state (columns are numbered from 1 to 7 in the visualization). The value displayed below represents the Minimax evaluation or, for MCTS, the estimated average reward.

The node colors distinguish the player whose turn comes next. The root represents the game state before the AI selects its move.

Note: The visualization may display only a subset of the explored nodes to keep the interface responsive. However, the reported statistics reflect the complete search.

Students are encouraged to use the playback controls after each AI move to understand how their implementations make decisions.

Understanding the Game Environment

Before implementing the search algorithms, it is important to understand how Connect Four is represented in the provided code. All game-related operations are implemented in `game.py`.

Game State and Players. The game board is represented by a 6×7 matrix, where each cell contains one of three possible values:

- `EMPTY = 0`: An empty cell.
- `HUMAN = 1`: A disc belonging to the human player.
- `AI = 2`: A disc belonging to the AI.

The human and AI alternate turns. The human plays first. A player wins by connecting four discs horizontally, vertically, or diagonally.

Actions and State Transitions. An action consists of choosing a column in which to drop a disc. Columns are indexed from 0 to 6 in Python.

The following functions define how the game evolves:

- `create_board()`: Creates an empty game board.

- `get_valid_moves(board)`: Returns the columns in which a disc can still be placed.
- `make_move(board, column, player)`: Returns a **new board** after placing the player's disc in the selected column. The original board is not modified.

These functions define the state transitions used by the search algorithms. Given a state s, an action a, and a player, the game engine determines the resulting state:

$$s' = T(s, a).$$

Terminal States. A state is terminal when a player has connected four discs or when the board is full.

The corresponding functions are:

- `check_winner(board, player)`: Checks whether a given player has won.
- `is_terminal(board)`: Checks whether the game has ended.
- `get_winner(board)`: Identifies the winner, if any.

Heuristic Evaluation Function. Minimax and Alpha-Beta cannot always explore the entire game tree. When the maximum search depth is reached, they use a **heuristic evaluation function** to estimate how favorable a position is for the AI.

The provided function is:

`evaluate_board(board, player=AI)`

It evaluates the board by examining groups of four consecutive cells (called *windows*) in all possible directions.

For the AI, the following scores are used:

Configuration	**Description**	**Score**
Center column	Each AI disc in the center column	+3
Four in a row	Four AI discs	+100
Three in a row	Three AI discs and one empty cell	+5
Two in a row	Two AI discs and two empty cells	+2
Opponent threat	Three opponent discs and one empty cell	−4

The scores are accumulated over the board. A positive evaluation indicates a favorable position for the AI, while a negative evaluation indicates a potentially dangerous one.

Terminal states are handled separately:

$$V(s) = \begin{cases} +10^6 & \text{if the AI wins,} \\ -10^6 & \text{if the human wins,} \\ 0 & \text{if the game is a draw.} \end{cases}$$

The large magnitude of terminal scores ensures that winning or losing takes priority over ordinary positional advantages.

Important: This evaluation function is a heuristic, not a perfect measure of the probability of winning. Different heuristics can lead to different decisions, even when the search depth remains unchanged.

Search Algorithms. The three algorithms implemented in this TD use the game environment differently:

- **Minimax** explores possible moves up to a specified depth, assuming that both players make optimal decisions according to the evaluation function.
- **Alpha-Beta Pruning** computes the same Minimax decision at a given depth while avoiding branches that cannot influence the final result.
- **MCTS** estimates the quality of moves through repeated simulations. Unlike the previous two algorithms, the provided MCTS implementation uses the outcomes of simulated games rather than the heuristic evaluation function.

Throughout the exercises, you will implement these algorithms without modifying the game rules or the evaluation function.

Exercise 1
— Minimax Algorithm

In this exercise, you will implement the **Minimax algorithm** to determine the best move for the AI in Connect Four.

Minimax explores possible future game states by alternating between two players:

- **MAX (AI):** Selects the move that maximizes the evaluation score.
- **MIN (Human):** Selects the move that minimizes the evaluation score.

Given a game state s and a remaining search depth d, Minimax is defined recursively as:

$$V(s,d) = \begin{cases} \text{Eval}(s), & \text{if } d = 0 \text{ or } s \text{ is terminal,} \\ \max\limits_{a \in A(s)} V(T(s,a), d-1), & \text{if MAX plays,} \\ \min\limits_{a \in A(s)} V(T(s,a), d-1), & \text{if MIN plays.} \end{cases}$$

Here, $A(s)$ denotes the set of valid moves, $T(s,a)$ the resulting state after applying action a, and $\text{Eval}(s)$ the evaluation function defined in `game.py`.

Open `algorithms.py` and locate the `minimax()` function.

1. **Terminal states and depth limit.**

   Implement the stopping condition of the recursive search. When the remaining depth reaches zero or the current state is terminal, return its evaluation using `evaluate_board()`.

2. **Generating successor states.**

   Retrieve the available actions using `get_valid_moves()`.

   For each valid action, generate the corresponding successor state using `make_move()` and recursively evaluate it with a decreased search depth.

   Make sure the player alternates between successive recursive calls.

3. **MAX and MIN decisions.**

   Implement the decision rule for each player:

   - When it is the AI's turn, retain the **maximum** value among the explored children.
   - When it is the human's turn, retain the **minimum** value among the explored children.

4. **Selecting the best move.**

   Complete the root search so that the AI selects the valid move associated with the highest Minimax value.

   Ensure that the function returns its result using the provided output structure.

5. **Testing your implementation.**

   Launch the web application:

   ```
   python app.py
   ```

   Select **Minimax** and play against the AI using different search depths (1, 2, and 4).

   Use the search-tree visualization to follow the recursive exploration and verify that the algorithm alternates correctly between MAX and MIN nodes.

Exercise 2
— Alpha-Beta Pruning

In this exercise, you will improve Minimax by implementing **Alpha-Beta Pruning**, which avoids exploring branches that cannot influence the final decision.

Alpha-Beta maintains two bounds throughout the search:

- α: The best value that MAX can guarantee along the current search path.
- β: The best value that MIN can guarantee along the current search path.

Initially, these bounds are set to:

$$\alpha = -\infty, \qquad \beta = +\infty.$$

During the recursive exploration, the bounds are updated according to the current player:

$$\boxed{\begin{array}{ll} \text{MAX node:} & \alpha \leftarrow \max(\alpha, v), \\ \text{MIN node:} & \beta \leftarrow \min(\beta, v), \end{array}}$$

where v is the best value found so far at the current node.

A branch can be pruned whenever:

$$\boxed{\alpha \geq \beta}$$

At this point, exploring additional children cannot change the decision made by the relevant ancestor.

Open `algorithms.py` and locate the `alphabeta()` function.

1. **Initialize the search bounds.**

   Initialize α and β to $-\infty$ and $+\infty$, respectively, at the root of the search.

   Ensure that both bounds are passed to the recursive search function.

2. **Implement the recursive search.**

   Reuse the Minimax logic from Exercise 1:

- Stop when the remaining depth reaches zero or the state is terminal.
- Generate successor states using `get_valid_moves()` and `make_move()`.
- Alternate between MAX and MIN at successive levels of the search tree.

3. **Update the bounds.**

   After evaluating each child:

   - At a MAX node, update the best value and the α bound.
   - At a MIN node, update the best value and the β bound.

   Pass the updated bounds to subsequent recursive calls.

4. **Implement pruning.**

   After updating the bounds, check whether $\alpha \geq \beta$.

   If this condition holds, stop exploring the remaining children of the current node.

   Use the provided tracing utilities to record pruning events so that they appear in the search-tree visualization.

5. **Select the best move.**

   Complete the root search to return the move with the highest Minimax value.

   Preserve the provided output structure, including the selected move, evaluation score, explored nodes, execution time, and search trace.

6. **Test your implementation.**

   Launch the web application and select **Alpha-Beta** as the AI algorithm.

   Test search depths 2, 4, and 6.

   Use the visualization to inspect the explored branches and identify where pruning occurs.

   Compare Alpha-Beta against Minimax at the **same search depth and game state**.

   Both algorithms should return the same Minimax value, while Alpha-Beta should generally explore fewer nodes.

Exercise 3
— Monte Carlo Tree Search (MCTS)

In this exercise, you will implement **Monte Carlo Tree Search (MCTS)**, a search algorithm that estimates the quality of actions through repeated simulations of the game.

Unlike Minimax and Alpha-Beta, MCTS does not explore the game tree up to a fixed depth. Instead, it incrementally constructs a search tree by repeating four phases:

1. **Selection:** Traverse the existing tree by selecting promising children.
2. **Expansion:** Add a new child corresponding to an unexplored action.
3. **Simulation:** Play a simulated game until a terminal state is reached.
4. **Backpropagation:** Update the statistics of the visited nodes using the simulation outcome.

Upper Confidence Bound for Trees (UCT).

During selection, MCTS balances *exploitation* (choosing actions with high estimated rewards) and *exploration* (choosing actions that have been tried less frequently).

For a child node i, the UCT score is:

$$\boxed{\mathrm{UCT}(i) = \underbrace{\overline{R}_i}_{\text{Exploitation}} + \underbrace{c\sqrt{\frac{\ln N}{n_i}}}_{\text{Exploration}}}$$

where:

- $\overline{R}_i$ is the average reward of child i, measured from the perspective of the player who selected the action leading to that child.
- N is the number of visits to the parent node.
- n_i is the number of visits to child i.
- $c > 0$ is the exploration constant.

A larger value of c encourages exploration, while a smaller value favors actions with high estimated rewards.

Open `algorithms.py` and locate the `mcts()` function and its associated helper functions.

1. **Implement UCT selection.**

   Complete the function responsible for computing the UCT score of a child node.

   Use the number of visits and accumulated rewards stored in the search tree.

   Handle unvisited children appropriately to ensure that unexplored actions can be selected.

2. **Implement tree selection.**

   Starting from the root, repeatedly select the child with the highest UCT score until reaching a node that is terminal or has unexplored actions.

   Use the existing tree structure to navigate between parent and child nodes.

3. **Implement expansion.**

   When the selected node has unexplored valid moves, choose one of them and create a new child node.

   Generate its game state using `make_move()` and update the tree structure.

   Ensure that each valid action is expanded at most once from a given node.

4. **Implement simulation (rollout).**

   Starting from the selected or newly expanded node, simulate a complete game by repeatedly choosing random valid moves.

   Continue until `is_terminal()` returns `True`.

   Determine the winner using `get_winner()`.

   In this implementation, use the following rewards:

$$R = \begin{cases} 1 & \text{if the player wins,} \\ 0.5 & \text{if the game ends in a draw,} \\ 0 & \text{if the player loses.} \end{cases}$$

5. **Implement backpropagation.**

   Starting from the node where the simulation began, traverse the tree back to the root.

   For every visited node:

   - Increment its visit count.
   - Update its accumulated reward using the simulation outcome.

- Evaluate the reward from the perspective of the player who made the move leading to that node.

Ensure that the statistics are updated consistently for both AI and human nodes.

6. **Repeat the search and select a move.**

   Combine selection, expansion, simulation, and backpropagation into the main MCTS loop.

   Repeat these phases for the specified number of simulations.

   Once all simulations are completed, select the root action whose child has the **highest visit count**.

   Return the selected move using the provided output structure, including the search statistics and visualization trace.

7. **Test your implementation.**

   Launch the web application and select **MCTS** as the AI algorithm.

   Test your implementation with 100, 500, and 2000 simulations.

   Use the search-tree visualization to inspect how the tree expands and how visit counts and estimated rewards evolve.

Exercise 4
— Comparing Search Algorithms

In the previous exercises, you implemented three game-search algorithms: Minimax, Alpha-Beta Pruning, and MCTS.

In this exercise, you will compare their computational performance and investigate how search parameters influence their behavior.

Performance metrics.

Each algorithm returns a dictionary containing the following information:

- `move`: The selected column.
- `score`: The evaluation of the selected move.
- `nodes_explored`: The number of explored nodes.
- `time`: The execution time in seconds.

Note that Minimax and Alpha-Beta return heuristic-based Minimax values, whereas MCTS returns a reward estimate obtained through simulations. Their scores are therefore not directly comparable.

Create a new Python file named `benchmark.py` inside the `code/` directory.

1. **Implement a benchmarking function.**

   Implement a function:

   ```
   benchmark(board, algorithm, **kwargs)
   ```

   The function should execute the specified algorithm on the given board and return a dictionary containing its name, selected move, evaluation score, number of explored nodes, and execution time.

   Use the existing implementations from `algorithms.py`.

2. **Compare Minimax and Alpha-Beta.**

   Using `create_board()`, generate an empty Connect Four board.

   Run Minimax and Alpha-Beta with the following search depths:

   $$d \in \{1, 2, 3, 4, 5\}.$$

   For each depth, record the selected move, evaluation score, number of explored nodes, and execution time.

   Display the results in a readable format.

   Verification: For every depth, both algorithms should return the same Minimax value. Alpha-Beta should explore no more nodes than Minimax.

3. **Compare different MCTS simulation budgets.**

   Run MCTS on the same initial board with:

   $$N \in \{100, 500, 1000, 2000\}.$$

   Record the selected move, estimated reward, number of explored nodes, and execution time.

   Use a fixed random seed, if supported by your implementation, to make the experiments reproducible.

4. **Automate the experiments.**

   Extend your benchmarking code to execute all configurations automatically.

   Organize the results into a table with the following columns:

Algorithm	**Configuration**	**Move**	**Nodes**	**Time (s)**
Minimax	Depth 1	—	—	—
Minimax	Depth 2	—	—	—
⋮	⋮	⋮	⋮	⋮
Alpha-Beta	Depth 1	—	—	—
⋮	⋮	⋮	⋮	⋮
MCTS	100 simulations	—	—	—
⋮	⋮	⋮	⋮	⋮

5. **Visualize the results.**

   Using `matplotlib`, generate two plots:

   - The number of explored nodes as a function of search depth for Minimax and Alpha-Beta.
   - The execution time as a function of the number of simulations for MCTS.

   Include appropriate axis labels and legends.